No Referee: Trustless Hunger Games for AI Agents

Information Security course project proposal · Oct 25, 2026 ·
@Tashi Technologies Corp (Pvt) Ltd

Summary

We will build a "Hunger Games" arena where 4–6 AI agents compete with no referee and no game server. Cryptography alone enforces the rules.

Every move is a signed Nostr event, moves stay hidden until every player has committed, and random events use randomness no single player can bias. We then attack our own protocol with agents that cheat and prompt-inject each other, and show that every cheat is caught and traced to its key.

The project extends topic 13 (commit-reveal rock-paper-scissors) into a full multi-party game, and uses topic 9 (Shamir secret sharing) to survive players who quit mid-game.

Problem and threat model

Online games and multi-agent systems normally trust a central server to enforce the rules. We remove that server: players distrust each other, and any of them may cheat.

- **Trusted:** the cryptographic primitives (SHA-256, Schnorr signatures on secp256k1) and each player's control of its own private key.
- **Untrusted relays:** Nostr relays may drop, delay or reorder events, but cannot forge signatures.
- **Malicious players may try to:** change a move after seeing others, bias random events, send different moves to different relays (equivocation), forge another player's move, quit mid-round to stall the game, and manipulate other agents through chat (prompt injection).
- **Out of scope:** hiding positions while proving facts about them (needs zero-knowledge proofs), network-level denial of service, and fake identities (the roster of public keys is fixed before the game starts).

Cryptographic design

Each attack in the threat model has a matching defence, mapped to course topics.

Attack	Defence	Course topic
Forging or denying a move	Every move is a Nostr event signed with the player's secp256k1 key (Schnorr)	Digital signatures
Changing a move after seeing others	Commit-reveal: publish H(move ‖ nonce ‖ round ‖ pubkey), reveal only once all commitments are in	13 Commitments
Biasing supply drops or random events	Each player commits to a random seed; the outcome is H(all revealed seeds)	13 Commitments
Quitting instead of revealing a seed (last-revealer attack)	Each seed is Shamir-shared (k-of-n) up front, so the remaining players rebuild a quitter's seed	9 Secret sharing
Equivocation across relays	Events go to several relays; two signed moves for the same round are proof of cheating	Digital signatures
Reading private alliance chat	End-to-end encrypted Nostr messages between agents (NIP-44)	Authenticated encryption
Disputing the result	Anyone replays the signed event log and gets the same winner; no referee needed	Verifiability

Red-team evaluation

We test the protocol by letting some agents cheat on purpose, then checking what it catches.

1. **Scripted cheaters:** one bot per attack in the table above. Success means every cheat is detected and traced to the correct public key, with no honest player falsely accused.

2. **Prompt injection:** agents negotiate alliances in chat, and attacker agents send messages such as fake "system" instructions. We measure how often each local model obeys them, then test defences such as a strict message format and labelling chat as untrusted input.
3. **Key claim:** injection can push an agent into a bad decision, but never into a rule-breaking move, because the protocol rejects anything not validly committed and signed.

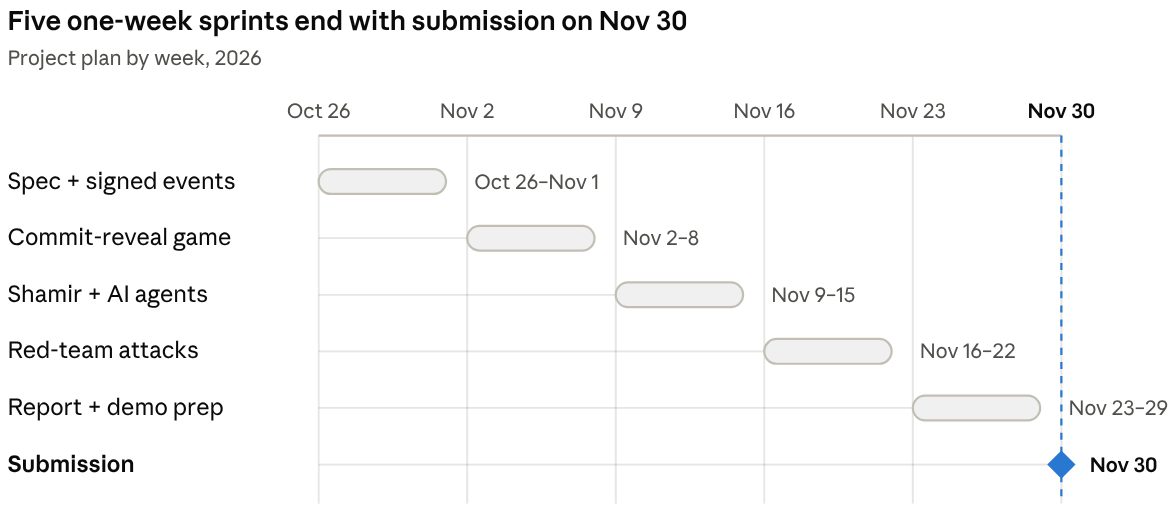
Game and implementation

- **Game:** an 8×8 grid, 4–6 tributes, a storm that shrinks the safe zone, random supply drops, about 20 rounds. Each round a tribute moves, attacks, hides or picks up an item. Last tribute standing wins.
- **Stack:** TypeScript with the open-source `nostr-tools` and `noble-curves` libraries, plus a self-hosted Nostr relay on one laptop for a reliable demo.
- **AI agents:** open models run locally through Ollama, one tribute per teammate's laptop, each a different model family. One or two tributes may use a free API tier (Groq or GitHub Models) in the final demo.
- **Budget:** $0. Every tool is open source or on a free tier.

Team and timeline

Each of the three members owns one layer, and each runs one AI tribute on their own laptop.

Member	Owns
Member 1	Crypto protocol: commitments, joint randomness, Shamir recovery, replay verifier
Member 2	Nostr networking, game engine, replay viewer
Member 3	AI agents: Ollama harness, cheating bots, prompt-injection experiments



project timeline · 5 weekly sprints to Nov 30

A full game runs with scripted bots by Nov 8, so AI agents and the red team build on a working protocol rather than racing the deadline.

Risks and deliverables

Risk	Mitigation
Small local models play slowly or badly	A short action menu per turn; scripted bots fill empty slots
Public relays are unreliable	A self-hosted relay for development and the demo
A player hands out invalid Shamir shares	Stretch goal: Feldman verifiable secret sharing
Scope creep	Hidden positions with proofs stay out of scope

Deliverables: the protocol library, the arena runner, the agent harness, a replay viewer that animates each game and flags caught cheaters, a short report with the attack results, and a live demo.

Open question for the instructor: does this count under topic 13, or as a custom topic?